

Campus Bulletin Board System

需求分析与项目计划

Group 10

浙江大学

2026 年 5 月 10 日

1 需求分析

2 项目计划

Background

- **目标用户**：在校大学生、教职工、校园社团组织者。
- **项目目标**：构建一个功能完善、易于维护的校园论坛，支持话题分区、互动讨论与内容管理。
- **预期收益**：提升校园信息流通效率，形成可检索的知识库，支持社团活动宣传与学术交流。

项目信息

- **项目名称:** Campus Bulletin Board System (校园论坛)
- **项目描述:** 面向大学校园的在线论坛系统
- **核心功能:** 用户注册、发帖、评论、点赞、通知、搜索、管理后台

需求描述

- UC-1 用户注册：访客通过用户名/邮箱/密码注册账号，需校验唯一性。
- UC-2 用户登录：支持账号密码登录，成功后返回 JWT Token。
- UC-3 密码重置：已登录用户可修改密码，旧密码需校验。
- UC-4 角色与权限：区分普通用户与管理员，接口级权限控制。

需求描述

- UC-5 板块浏览：按排序展示板块列表，支持查看板块下帖子。
- UC-6 发帖：登录用户在指定板块发布帖子，支持富文本/草稿。
- UC-7 帖子管理：作者可编辑/删除自己的帖子；管理员可置顶、加精、隐藏。
- UC-8 列表查询：支持分页、排序（最新/最热），返回帖子摘要。

功能性需求——评论、互动与通知

评论与互动

- UC-9 评论发布与回复（楼中楼）
- UC-10 点赞/取消点赞（帖子、评论）
- UC-11 实时计数聚合（点赞数、评论数）

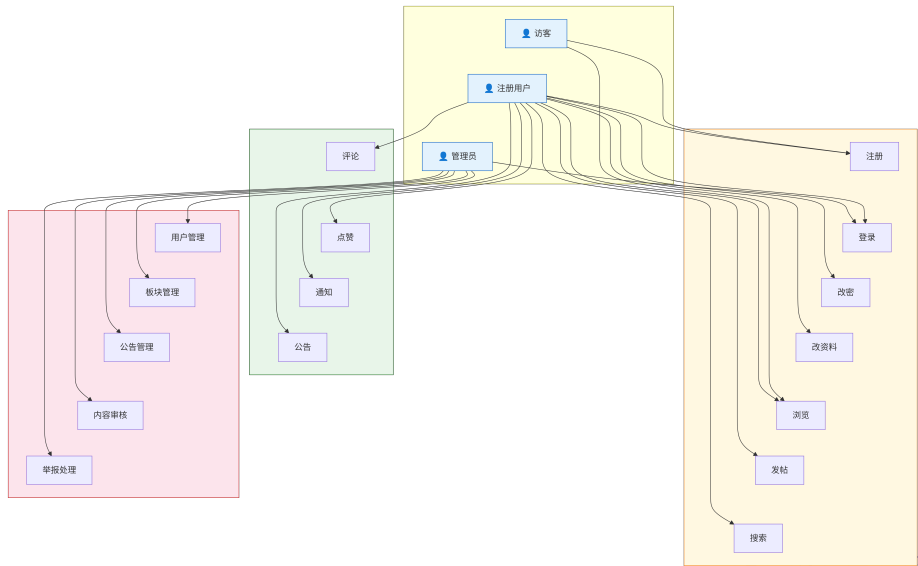
通知与消息

- UC-12 评论/回复/点赞触发通知
- UC-13 未读计数与已读标记
- UC-14 系统公告与站内信

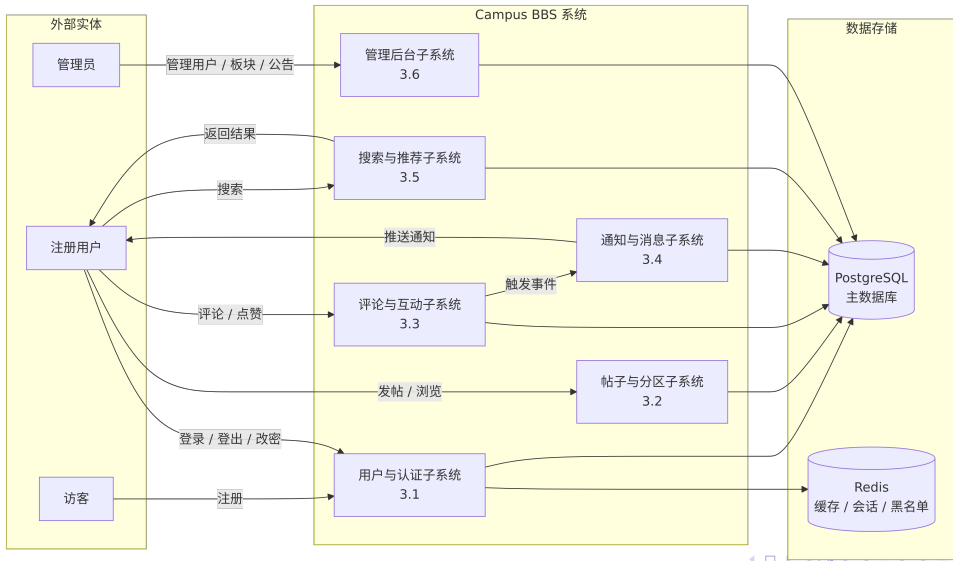
需求描述

- UC-15 用户管理：管理员可查看用户列表、封禁/解封、调整角色。
- UC-16 板块/公告管理：CRUD 板块，发布/编辑/下架系统公告。
- UC-17 内容审核：对帖子/评论进行隐藏/恢复/删除（软删除）。
- UC-18 举报处理：查看举报列表，处理并记录。

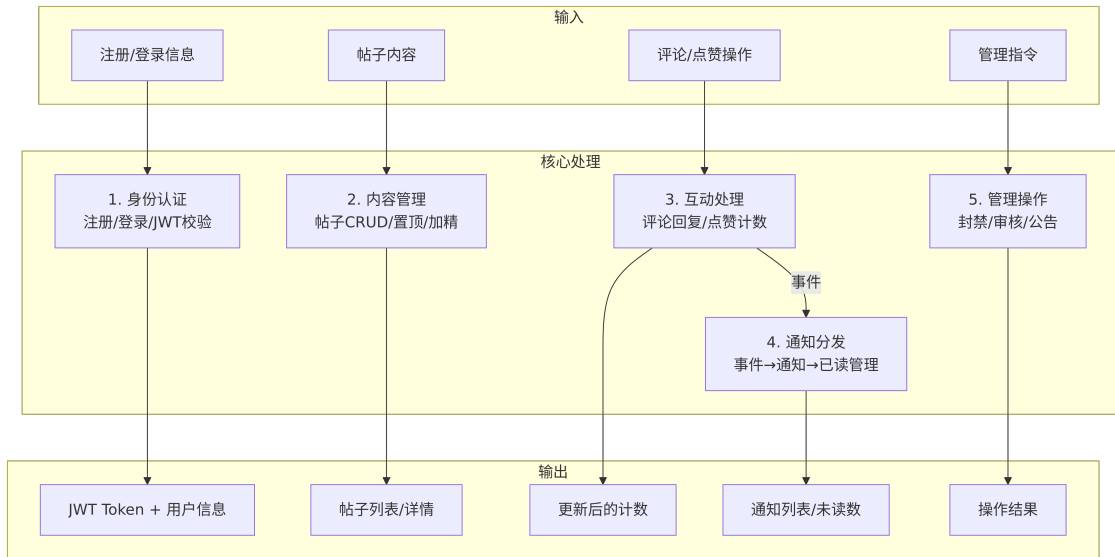
用例图



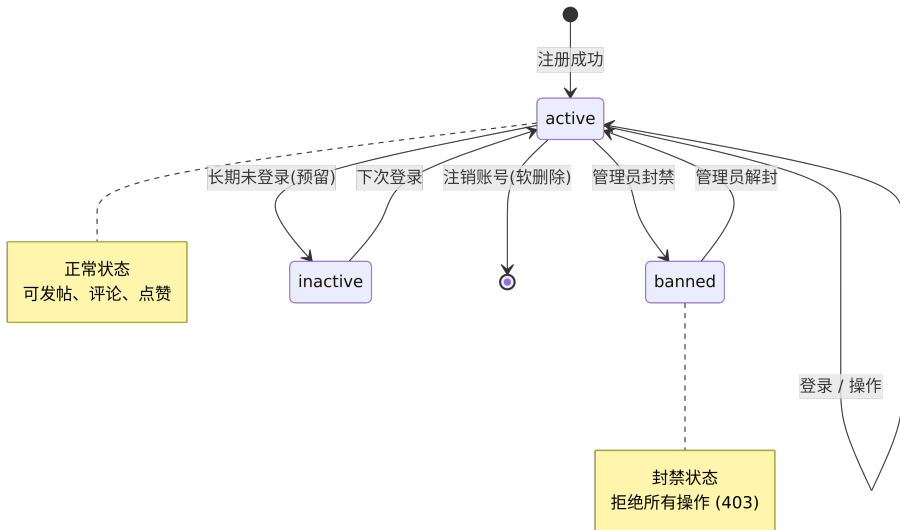
系统上下文数据流图



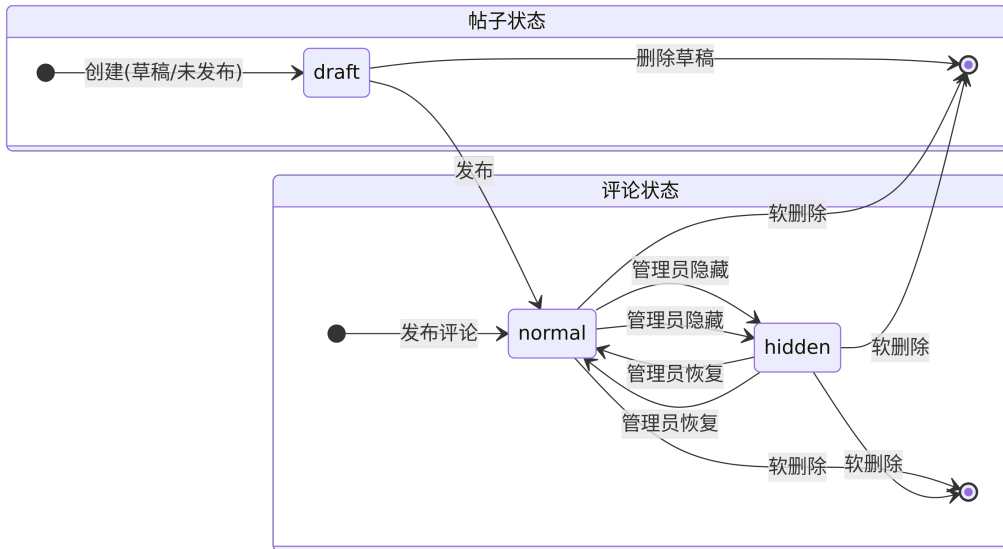
第 0 层数据流图



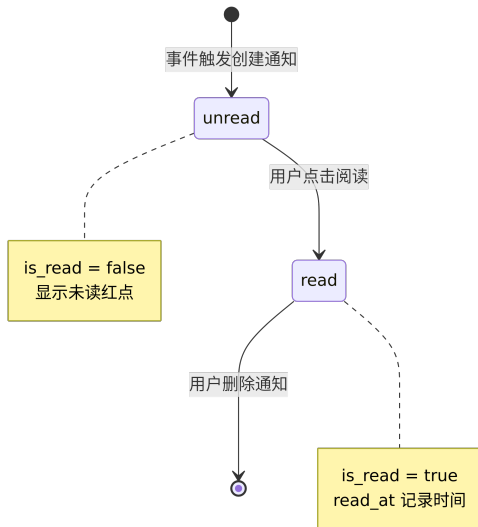
状态图——用户生命周期



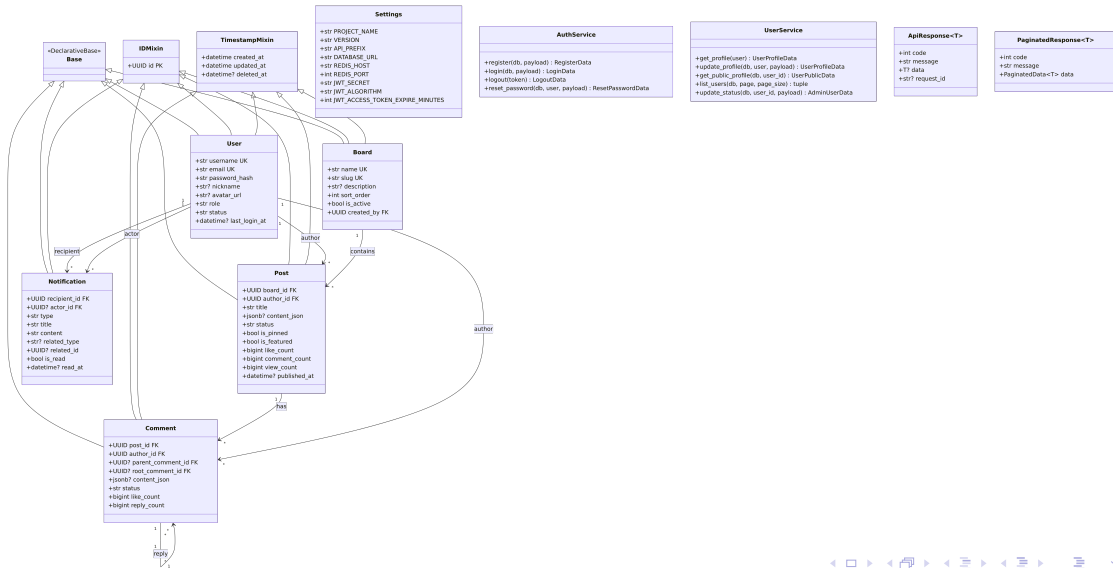
状态图——帖子与评论



状态图——通知生命周期



类图——后端核心模型



CRC Cards ——核心类 (1/2)

User	
Responsibilities	Collaborators
维护用户基本信息（用户名、邮箱、密码哈希）	AuthService
记录角色（user/admin）与状态（active/banned）	Post, Comment
维护最后登录时间	Notification

Post	
Responsibilities	Collaborators
存储帖子标题、内容、状态	User（作者）
维护置顶/加精标记	Board（所属板块）
聚合点赞数、评论数、浏览数	Comment, PostLike

CRC Cards ——核心类 (2/2)

Comment	
Responsibilities	Collaborators
存储评论内容、状态 支持楼中楼回复 (parent/root) 聚合点赞数、回复数	User (作者), Post Comment (自身) CommentLike

Notification	
Responsibilities	Collaborators
记录通知类型 (comment/reply/like/system) 维护已读状态与已读时间	User (接收人/触发人) Post, Comment (关联对象)

Board	
Responsibilities	Collaborators
存储板块名称、slug、描述、排序值 维护板块启用状态 (is_active) 支持按 sort_order 排序展示	User (创建人) Post (所含帖子) AdminPanel (管理操作)

AuthService	
Responsibilities	Collaborators
用户注册（校验唯一性、密码哈希）	User, security.py
用户登录（校验密码、生成JWT）	User, Redis（Token 黑名单）
用户登出（Token 加入黑名单）	Redis
密码重置（校验旧密码、更新哈希）	User, security.py

UserService	
Responsibilities	Collaborators
获取/更新当前用户资料	User
获取用户公开资料	User
管理员：用户列表分页查询	User
管理员：更新用户状态（封禁/解封/角色）	User, Notification

安全性分析

认证与授权安全

- 密码使用 Argon2 哈希存储 (pwdlib)
- JWT Token (HS256) 设有效期, 支持黑名单登出
- 接口级 RBAC 权限控制 (user/admin)

数据与传输安全

- 核心业务数据软删除 (deleted_at), 可恢复
- SQLAlchemy ORM 防 SQL 注入
- 预留 HTTPS 传输加密部署方案

接口安全

- 登录/注册接口速率限制 (slowapi), 防暴力破解
- 输入参数 Pydantic 校验, 防畸形数据

性能需求

响应时间指标

- 帖子列表接口响应时间 < 200ms
- 数据库单表分页查询 < 100ms
- 登录/注册接口 < 300ms

数据库优化

- 索引优化 (created_at、board_id、user_id)
- 分页查询避免深页性能衰减
- 计数字段冗余, 减少实时聚合

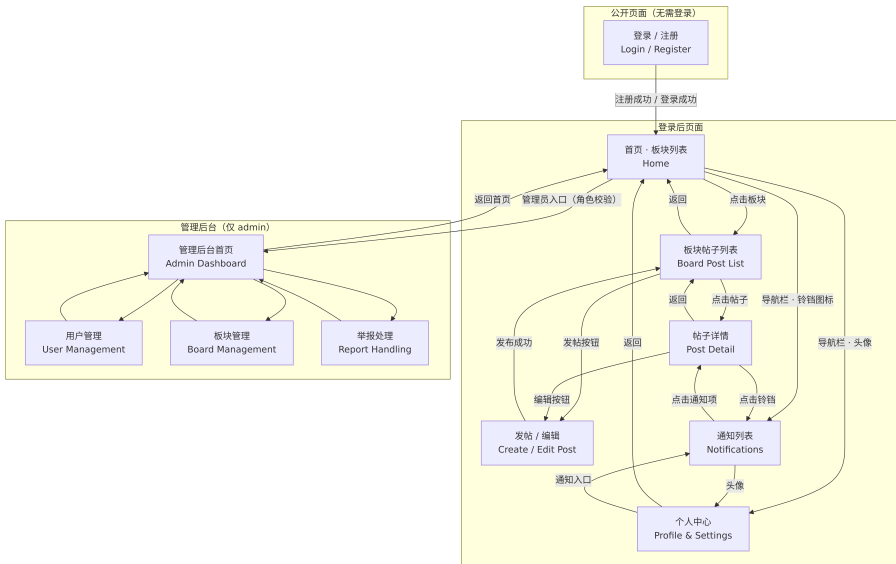
并发与容量

- 支持并发用户 500+ (MVP 阶段)
- 图片/附件预留对象存储接口
- Redis 缓存热点数据 (Token、计数)

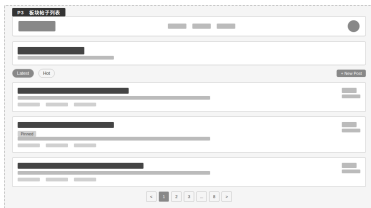
可用性目标

- 系统可用性目标 99.9%
- 核心业务数据软删除, 可恢复
- 健康检查端点 (/health) 用于监控

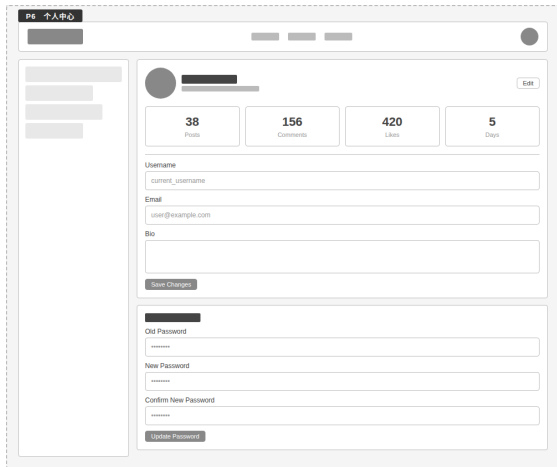
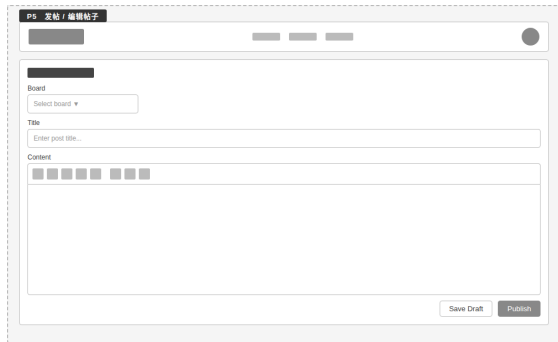
UI 设计——页面流程图



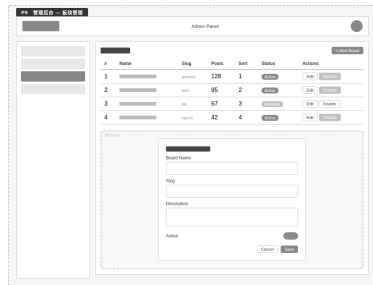
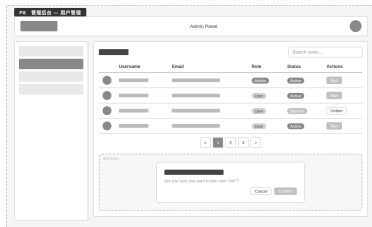
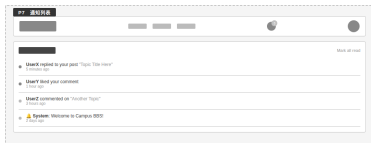
UI 设计——核心页面线框图 (1/3)



UI 设计——核心页面线框图 (2/3)



UI 设计——核心页面线框图 (3/3)



文档体系

- **DesignDiagrams.md**: UML 类图、用例图、状态图等
- **DatabaseDesign.md**: 数据库表结构设计与 ER 图
- **DevelopmentSpecification.md**: 代码规范、API 设计规范
- **ProjectRequirements.md**: 功能性与非功能性需求

质量保障措施

- API 文档采用 FastAPI 自动生成的交互式文档
- 文档随迭代同步更新，版本控制入 Git
- 代码提交前通过 black + ruff 自动格式化
- PR 必须经过代码审查与文档核对

项目范围

MVP 必做范围

- ① 用户与认证子系统 (Part1)
- ② 帖子与分区子系统 (Part2)
- ③ 评论与互动子系统 (Part3)
- ④ 通知基础子系统 (Part4 基础)
- ⑤ 管理后台基础 (Part6 基础)

迭代扩展范围

- 迭代一：系统公告与站内信；公告管理、举报处理
- 迭代二：关键词搜索、热门排序
- 迭代三：个性化推荐；操作审计与日志

范围约束

- 不实现即时聊天 (IM)；不实现第三方支付；图片/附件仅预留接口，MVP 阶段不接入真实对象存储

迭代式开发模型 (Iterative Model)

- 采用短周期迭代，每个迭代 1 周，产出可演示的增量版本。
- MVP 阶段聚焦核心闭环（注册 → 发帖 → 评论 → 通知）。
- 每轮迭代结束进行小组 Review 与需求微调。

迭代式开发模型 (Iterative Model)

- 采用短周期迭代，每个迭代 1 周，产出可演示的增量版本。
- MVP 阶段聚焦核心闭环（注册 → 发帖 → 评论 → 通知）。
- 每轮迭代结束进行小组 Review 与需求微调。

敏捷实践

- Git 分支管理：feature → develop → main，PR 代码审查。
- 持续集成：Husky pre-commit (black + ruff)，pytest 自动测试。
- 文档驱动：API 文档与数据库设计文档随迭代同步更新。

Scale Estimation —— 规模估算

功能点估算

- 用户认证模块: ~8 FP
- 帖子/分区模块: ~10 FP
- 评论/互动模块: ~8 FP
- 通知模块: ~6 FP
- 管理后台: ~8 FP
- 搜索推荐 (迭代): ~6 FP

代码规模估算

- 后端 (Python): ~4,000–5,000 LOC
- 前端 (TS): ~6,000–8,000 LOC
- 测试代码: ~1,500–2,000 LOC
- 文档/配置: ~2,000 LOC

工期估算

- 总工期约 6 周 (MVP 3 周 + 迭代一 1 周 + 迭代二 1 周 + 迭代三 1 周缓冲)。
- 团队规模 6 人, 总工作量约 216 人时 (按每人每周 6 小时投入估算)。

资源配置

人力资源

- 组长 1 人：架构设计、代码审核、Git 管理
- 前端 2 人：用户端 / 管理端界面开发
- 后端 2 人：业务接口与数据库开发
- 测试兼文档 1 人：测试用例、CI/CD、文档维护

软硬件环境

- 开发环境：Python 3.14 + FastAPI + PostgreSQL + Redis
- 前端：Vue + TypeScript + pnpm
- 容器化：Docker Compose 一键启动依赖服务
- 版本控制：Git + Husky hooks (lint-staged)

工具链

- 格式化：black；代码检查：ruff；测试：pytest；包管理：uv（后端） / pnpm（前端）

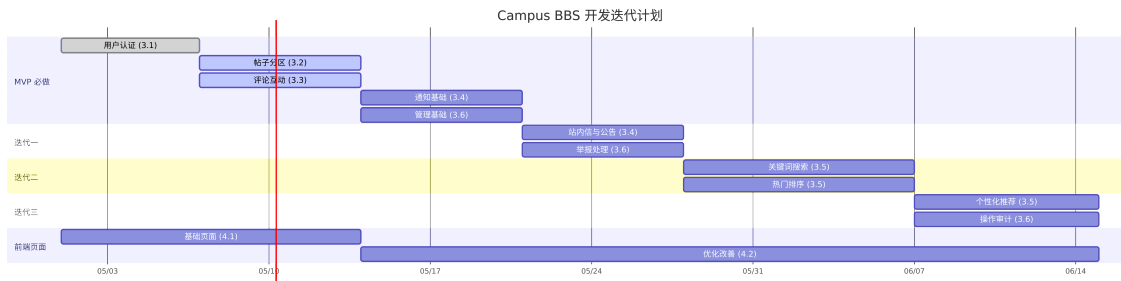
交付物定义

阶段	交付物	说明
MVP	源代码	后端核心接口 + 前端基础页面
MVP	数据库迁移脚本	Alembic 迁移文件
MVP	API 文档	FastAPI 自动生成交互式文档
迭代一	管理后台	用户/板块/公告管理功能
迭代一	通知系统	站内信与系统公告
迭代二	搜索功能	关键词搜索与热门排序
迭代三	推荐模块	个性化推荐（可选）
迭代三	审计日志	操作审计与日志查询
结项	设计文档	UML 图、数据库设计、需求文档
结项	测试报告	单元测试、集成测试报告

任务与计划——功能优先级与迭代规划

优先级	子系统	说明
MVP 必做	Part1、Part2、Part3	用户认证、帖子分区、评论互动
MVP 必做	Part4（基础）	评论/回复/点赞通知、未读计数
MVP 必做	Part6（基础）	用户管理、板块管理
迭代一	Part4（完整）	系统公告与站内信
迭代一	Part6（完整）	公告管理、举报处理
迭代二	Part5	关键词搜索、热门排序
迭代三	Part5（进阶）	个性化推荐
迭代三	Part6（进阶）	操作审计与日志

任务与计划——功能迭代路线图



风险项	概率	影响	应对措施
组员时间冲突	中	高	Issue 驱动, 组员承接任务, 灵活安排时间
数据库性能瓶颈	低	中	索引优化 + 分页 + 预留 Redis 缓存扩展
需求变更	中	中	迭代规划预留 20% 缓冲时间, 优先保障 MVP

代码文档

- 接口文档: FastAPI 自动生成, 随代码同步更新
- 核心函数添加 docstring, 说明参数与返回值
- 复杂业务逻辑添加行内注释

设计文档

- 需求变更必须同步更新
ProjectRequirements.md
- 数据库变更必须同步更新
DatabaseDesign.md 并生成迁移脚本
- UML 图使用 Mermaid 绘制, 版本入 Git

质量检查清单

- PR 合并前: 代码审查通过、测试通过、文档已更新、black + ruff 检查通过

总结与下一步

当前进展

- 用户与认证子系统 (Part1) 已实现
- 帖子分区 (Part2)、评论互动 (Part3)、通知 (Part4)、管理后台 (Part6) 路由已定义
- 前端页面线框图设计完成
- 数据库迁移与基础架构搭建完成

总结与下一步

当前进展

- 用户与认证子系统 (Part1) 已实现
- 帖子分区 (Part2)、评论互动 (Part3)、通知 (Part4)、管理后台 (Part6) 路由已定义
- 前端页面线框图设计完成
- 数据库迁移与基础架构搭建完成

下一步计划

- 完成 MVP 阶段各子系统的核心接口实现
- 推进前端页面开发与管理端界面
- 编写测试用例, 配置 CI/CD 流程

感谢聆听！

Q & A